Hello World 예제 상세 분석

개요

이 문서는 Proof of SQL의 `hello_world` 예제를 라인별로 상세히 분석하여 영지식 증명 기반 SQL 시스템의 전체적인 동작 방식을 이해할 수 있도록 합니다.

분석 대상 파일: `crates/proof-of-sql/examples/hello_world/main.rs`

예제 목적: `SELECT b FROM table WHERE a = 2` 쿼리에 대한 영지식 증명 생성 및 검증

테스트 데이터

```
| a | b |
|-----|---------|
| 1 | hi |
| 2 | hello |
| 3 | there |
| 2 | world |
```

예상 결과: a=2인 행들의 b 컬럼 값 → `["hello", "world"]`

코드 라인별 상세 분석

1. 문서 및 Import 섹션

```
#![doc = include_str!("README.md")]
```

한국어 설명:

- 이 라인은 README.md 파일의 내용을 현재 모듈의 문서로 포함시킵니다
- Rust의 문서화 시스템을 활용하여 예제 설명을 코드와 함께 관리합니다

```
use ark_std::test_rng;
```

한국어 설명:

- `ark_std` 는 arkworks 암호학 라이브러리의 표준 유틸리티입니다
- `test_rng` 는 테스트용 난수 생성기로, 암호학적 파라미터 생성에 사용됩니다
- 실제 운영 환경에서는 보안이 강화된 난수 생성기를 사용해야 합니다

```
use proof_of_sql::{
base::database::{
owned_table_utility::{bigint, owned_table, varchar},
OwnedTableTestAccessor, TableRef, TestAccessor,
},
proof_primitive::dory::{
DynamicDoryEvaluationProof, ProverSetup, PublicParameters, VerifierSetup,
},
sql::{parse::QueryExpr, proof::VerifiableQueryResult},
};
```

한국어 설명:

- **데이터베이스 관련 import**:
- `bigint` , `varchar` : 데이터 타입 생성 헬퍼 함수들
- `owned_table` : 테이블 생성 유틸리티
- `OwnedTableTestAccessor` : 테스트용 데이터베이스 접근자
- `TableRef` : 테이블 참조 구조체
- **암호학적 프리미티브 import**:
- `DynamicDoryEvaluationProof` : DORY commitment 스킴 기반 증명 시스템
- `ProverSetup` : 증명자(Prover) 설정 구조체
- `PublicParameters` : 공개 파라미터 (신뢰할 수 있는 설정)
- `VerifierSetup` : 검증자(Verifier) 설정 구조체
- **SQL 처리 관련 import**:
- `QueryExpr` : SQL 쿼리 표현식 파서
- `VerifiableQueryResult` : 검증 가능한 쿼리 결과

```
use std::{
io::{stdout, Write},
time::Instant,
};
```

한국어 설명:

- 표준 라이브러리에서 입출력과 시간 측정 기능을 import합니다
- `stdout` , `Write` : 콘솔 출력을 위한 인터페이스
- `Instant` : 고정밀 시간 측정을 위한 구조체

2. 유틸리티 함수들

```
/// # Panics
///
/// Will panic if flushing the output fails, which can happen due to issues with
fn start_timer(message: &str) -> Instant {
print!("{message}...");
stdout().flush().unwrap();
Instant::now()
}
```

한국어 설명:

- **목적**: 작업 시작 시점을 기록하고 진행 상황을 출력하는 타이머 시작 함수
- **매개변수**: `message` - 출력할 작업 설명 메시지
- **반환값**: `Instant` - 현재 시점의 타임스탬프
- **동작 과정**:

1. `print!("{message}...")` : 작업 메시지를 출력 (줄바꿈 없이)
2. `stdout().flush().unwrap()` : 출력 버퍼를 강제로 비워서 즉시 화면에 표시
3. `Instant::now()` : 현재 시점을 기록하여 반환

- **패닉 조건**: 출력 스트림에 문제가 있어 flush가 실패할 경우

```
/// # Panics
///
```

```
/// This function does not panic under normal circumstances but may panic if the
fn end_timer(instant: Instant) {
println!(" {:?}", instant.elapsed());
}
```

한국어 설명:

- **목적**: 작업 완료 시점에서 경과 시간을 계산하고 출력하는 타이머 종료 함수
- **매개변수**: `instant - start_timer` 에서 반환된 시작 시점
- **동작 과정**:

1. `instant.elapsed()` : 시작 시점부터 현재까지의 경과 시간 계산
2. `println!(" {:?}", ...)` : 경과 시간을 디버그 형식으로 출력 (예: “520.959485ms”)

- **패닉 조건**: 일반적으로 패닉하지 않지만, 내부 출력 과정에서 문제가 발생할 수 있음

3. 메인 함수 - 전체 증명 플로우

```
/// # Panics
///
/// - Will panic if the GPU initialization fails during `init_backend`.
/// - Will panic if the table reference cannot be parsed in `add_table`.
/// - Will panic if the offset provided to `add_table` is invalid.
/// - Will panic if the query string cannot be parsed in `QueryExpr::try_new`.
/// - Will panic if the table reference cannot be parsed in `QueryExpr::try_new`
/// - Will panic if the query expression creation fails.
/// - Will panic if printing fails during error handling.
```

```
fn main() {
```

한국어 설명:

- **목적**: Proof of SQL의 전체 워크플로우를 보여주는 메인 함수
- **패닉 조건들**: 다양한 초기화 및 처리 단계에서 발생할 수 있는 오류 상황들을 명시

3.1 GPU 초기화 (선택적)

```
#[cfg(feature = "blitzar")]
{
let timer = start_timer("Warming up GPU");
proof_of_sql::base::commitment::init_backend();
end_timer(timer);
}
```

한국어 설명:

- **조건부 컴파일**: `blitzar` 피처가 활성화된 경우에만 실행
- **목적**: GPU 가속을 위한 Blitzar 백엔드 초기화
- **동작 과정**:

1. GPU 워밍업 타이머 시작
2. `init_backend()` : CUDA 커널 로딩 및 GPU 메모리 할당
3. 초기화 시간 측정 및 출력 (일반적으로 500ms 내외)

- **성능 영향**: GPU 가속을 사용하면 대용량 데이터 처리 시 10-100배 성능 향상

3.2 암호학적 설정 초기화

```
let timer = start_timer("Loading data");

let public_parameters = PublicParameters::test_rand(5, &mut test_rng());

let prover_setup = ProverSetup::from(&public_parameters);

let verifier_setup = VerifierSetup::from(&public_parameters);
```

한국어 설명:

- **목적**: 영지식 증명 시스템을 위한 암호학적 파라미터 설정
- **동작 과정**:

1. `PublicParameters::test_rand(5, &mut test_rng())`:
- 테스트용 공개 파라미터 생성
- `5` 는 보안 파라미터 (실제로는 더 큰 값 사용)
- 타원곡선 점들과 페어링 파라미터 포함
2. `ProverSetup::from(&public_parameters)`:
- 증명자가 사용할 설정 생성
- commitment key와 증명 생성용 파라미터 포함
3. `VerifierSetup::from(&public_parameters)`:
- 검증자가 사용할 설정 생성
- 검증용 키와 파라미터만 포함 (더 가벼움)

3.3 데이터베이스 설정 및 테이블 생성

```
let  mut accessor =

OwnedTableTestAccessor::<DynamicDoryEvaluationProof>::new_empty_with_setup(&prov
```

한국어 설명:

- **목적**: 테스트용 데이터베이스 접근자 생성
- **타입 파라미터**: `DynamicDoryEvaluationProof` - 사용할 증명 시스템 지정
- **설정**: `prover_setup` 을 사용하여 commitment 생성 준비

```
accessor.add_table(
TableRef::new("sxt", "table"),
owned_table([
bigint("a", [1, 2, 3, 2]),
varchar("b", ["hi", "hello", "there", "world"]),
]),
0,
);
```

한국어 설명:

- **목적**: 테스트 데이터가 포함된 테이블을 데이터베이스에 추가
- **매개변수 분석**:

1. `TableRef::new("sxt", "table")` :

- 스키마명: "sxt" (Space and Time의 줄임말)
- 테이블명: "table"

2. `owned_table([...])` : 테이블 데이터 정의

- `bigint("a", [1, 2, 3, 2])` : 정수형 컬럼 'a' 생성
- `varchar("b", ["hi", "hello", "there", "world"])` : 문자열 컬럼 'b' 생성

3. `0` : 테이블의 오프셋 (시작 인덱스)

- **내부 동작**:
- 각 컬럼에 대해 암호학적 commitment 생성
- 데이터 무결성 보장을 위한 해시 계산

```
end_timer(timer);
```

한국어 설명: 데이터 로딩 시간 측정 완료 (일반적으로 3ms 내외)

3.4 SQL 쿼리 파싱

```
let timer = start_timer("Parsing Query");
let query = QueryExpr::try_new(
"SELECT b FROM table WHERE a = 2".parse().unwrap(),
"sxt".into(),
&accessor,
)
.unwrap();
end_timer(timer);
```

한국어 설명:

- **목적**: SQL 쿼리를 내부 표현으로 파싱하고 검증
- **동작 과정**:

1. `"SELECT b FROM table WHERE a = 2".parse().unwrap()`:

- SQL 문자열을 sqlparser를 사용하여 AST로 파싱
- 문법 오류 시 패닉 발생

2. `QueryExpr::try_new(ast, "sxt".into(), &accessor)`:

- AST를 Proof of SQL의 내부 표현으로 변환
- 스키마명 "sxt" 지정
- `accessor` 를 통해 테이블 메타데이터 검증
- 타입 검사 및 컬럼 존재 여부 확인
- **결과**: 증명 가능한 형태의 쿼리 표현식 생성
- **처리 시간**: 일반적으로 1-2ms

3.5 영지식 증명 생성

```
let timer = start_timer("Generating Proof");
let verifiable_result = VerifiableQueryResult::<DynamicDoryEvaluationProof>::new
query.proof_expr(),
&accessor,
&&prover_setup,
&[],
)
.unwrap();
end_timer(timer);
```

한국어 설명:

- **목적**: 쿼리 실행 결과에 대한 영지식 증명 생성 (핵심 단계!)
- **매개변수 분석**:

1. `query.proof_expr()` : 파싱된 쿼리의 증명 가능한 표현식

2. `&accessor` : 데이터베이스 접근자 (실제 데이터 포함)

3. `&&prover_setup` : 증명자 설정 (이중 참조)

4. `&[]` : 추가 컨텍스트 (빈 배열)

- **내부 동작 과정**:

1. **쿼리 실행**: WHERE 조건에 맞는 행 필터링

2. **다항식 변환**: SQL 연산을 다변수 다항식으로 변환

3. **Sumcheck 프로토콜**: 다항식 합의 정확성 증명

4. **Commitment 생성**: 결과 데이터에 대한 암호학적 commitment

5. **증명 패키징**: 모든 증명 요소를 하나의 구조체로 결합

- **처리 시간**: GPU 사용 시 400-500ms, CPU만 사용 시 더 오래 걸림

3.6 증명 검증

```
let timer = start_timer("Verifying Proof");

let result = verifiable_result.verify(query.proof_expr(), &accessor, &&verifier_

end_timer(timer);
```

한국어 설명:

- **목적**: 생성된 증명의 유효성을 검증하고 쿼리 결과 추출

- **매개변수**: 증명 생성과 동일하지만 `verifier_setup` 사용

- **내부 검증 과정**:

1. **Sumcheck 검증**: 다항식 합의 정확성 확인

2. **Commitment 검증**: 데이터 무결성 확인

3. **쿼리 로직 검증**: SQL 연산의 정확성 확인

4. **결과 추출**: 검증된 쿼리 결과 반환

- **처리 시간**: 일반적으로 7ms 내외 (검증은 증명보다 훨씬 빠름)

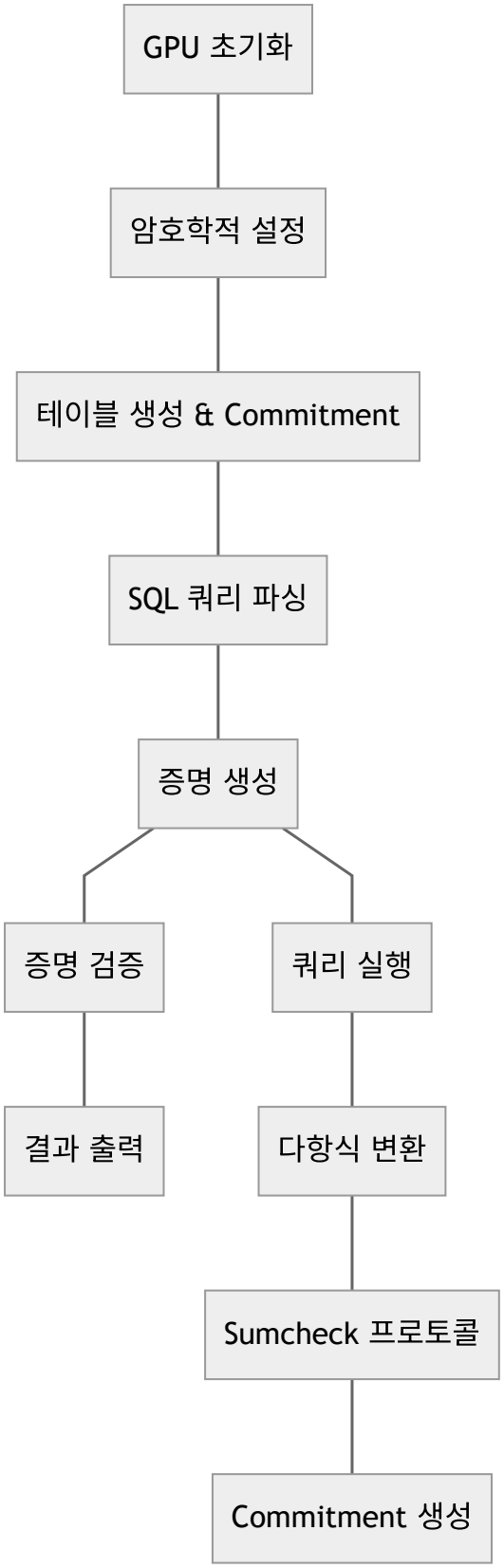
3.7 결과 출력

```
match result {
Ok(result) => {
println!("Valid proof!");
println!("Query result: {:?}", result.table);
}
Err(e) => {
println!("Error: {e:?}");
}
}
```

한국어 설명:

- **성공 케이스**:
- "Valid proof!" 메시지 출력
- 쿼리 결과 출력: `{b: VarChar(["hello", "world"])}`
- **실패 케이스**: 오류 정보 출력
- **예상 결과**: a=2인 행들의 b 컬럼 값들이 올바르게 출력됨

전체 워크플로우 요약



성능 특성

| 단계 | 일반적인 소요 시간 | 설명 |

|------|------------------|------|

| GPU 초기화 | ~520ms | 최초 1회만 필요 |

| 데이터 로딩 | ~3ms | 테이블 크기에 비례 |

| 쿼리 파싱 | ~2ms | 쿼리 복잡도에 비례 |

| 증명 생성 | ~467ms | 가장 비용이 큰 단계 |

| 증명 검증 | ~7ms | 증명보다 훨씬 빠름 |

핵심 개념 정리

1. **영지식 증명**: 실제 데이터를 노출하지 않고 쿼리 결과의 정확성만 증명
2. **Commitment**: 데이터의 암호학적 지문, 변조 감지 가능
3. **Sumcheck 프로토콜**: 다항식 합의 정확성을 효율적으로 검증하는 방법
4. **DORY**: 고성능 commitment 스킴, GPU 가속 지원
5. **Verifiable Query Result**: 증명과 결과가 함께 패키징된 구조체

이 예제는 Proof of SQL의 전체적인 동작 방식을 보여주는 완벽한 시작점입니다. 복잡한 암호학적 세부사항은 내부적으로 처리되며, 개발자는 간단한 API를 통해 영지식 증명 기반 SQL을 사용할 수 있습니다.